Supersede: when RAG cites the obsolete version

Measuring staleness in RAG, and a librarian agent that recovers document lineage

Carl Hatoum

Living draft. Every number is regenerated from `results/` (revision `ee31b19+dirty`, 2026-11-05).

Abstract

Document collections keep old versions next to the documents that replace them, and retrieval-augmented generation (RAG) treats every version as equally valid. Using the IETF RFC series, whose index records which RFC replaces which, we build a benchmark of 9,852 documents and 1,543 replacement links. Lexical, dense and hybrid retrievers all rank the obsolete version first for a large share of queries, and the error reaches generated answers when version metadata is missing. Known lineage fixes both; unknown lineage must be inferred. We infer it with a cascade: a cheap feature-based scorer for clear cases, and a tool-using *librarian agent* for the uncertain ones.

Key findings

1. **Retrieval prefers the obsolete version.** For questions whose answer changed between versions, the hybrid retriever puts the obsolete document first 65% of the time; a better retriever is *more* stale, not less (Fig. 2).
2. **It reaches answers when metadata is missing.** Without dates or version numbers in the context, 21.7% of answers are stale; showing both versions with the old one marked brings this to 0.0% (Fig. 3).
3. **Lineage can be partly inferred cheaply.** A logistic regression on similarity and date features closes 51% of the gap to perfect lineage, with every explicit cue removed (Fig. 4).
4. **An agent helps where the scorer is unsure** *(preliminary, n=20).* On pairs the scorer finds ambiguous, a local 9.7B-parameter agent reaches 0.90 balanced accuracy against 0.65 for the scorer. Whether the model follows the investigation procedure matters (Fig. 6).

1 The problem

Standards are revised, procedures reissued, policies replaced, and the old version usually stays in the collection. Nothing in a RAG pipeline [1] tells the retriever which version is in force, and the old document is often the *better* match: it has the canonical title and states the topic plainly, while the newer one may describe its changes. Figure 1 shows a real case from our benchmark.

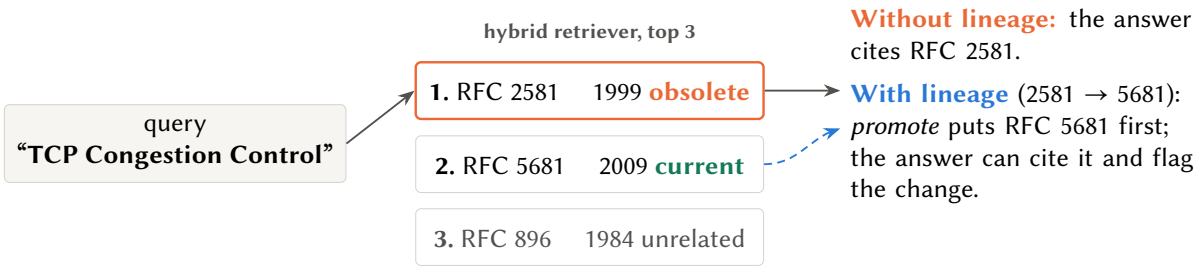


Figure 1. A real ranking from the benchmark. RFC 5681 replaced RFC 2581 in 2009, but the hybrid retriever ranks the obsolete document first. The same happens for TLS 1.2: the current version, RFC 9846, only comes fourth, behind two documents it replaced.

We ask three questions:

Q1. How often does retrieval surface the obsolete version first? (§3)

Q2. Does it reach the generated answer, and can an answer policy contain it? (§4)

Q3. When lineage is not recorded, can we infer it, cheaply and with an agent? (§5–6)

How to read the numbers

stale@1	share of queries whose *top* result is an obsolete version (lower is better).
current@1	share of queries whose top result is a version in force (higher is better).
lineage	the graph of which document replaces which. *Oracle* lineage is the true one, from the index.
gap closed	how much of the improvement from no lineage to oracle lineage an inferred lineage recovers.
balanced accuracy	the mean of the hit rate on true replacements and on non-replacements; 0.5 is chance.
[a, b]	a 95% bootstrap confidence interval. Overlapping intervals mean the difference is not established.

2 The benchmark

final · RFC snapshot e1669ddd5a87

Documents and lineage. We parse the RFC index (`rfc-index.xml`, built 2026-10-30): 9,852 RFCs, each with a title, year, keywords and, for 9,145 of them, an abstract. The index records *obsoletes* and *obsoleted-by*; taking a link from either side gives 1,543 replacement links over 1,384 obsolete documents, forming 800 lineages (the largest has 35 documents). The *current versions* of a document are the ends of its chain of replacements; a split can produce several. *Updates* relations (2,162) amend a document without replacing it and are not used as ground truth.

Splits. All queries and pairs of one lineage share a split (30% dev, 70% test), so no lineage is seen on both sides.

Title queries. For each link, the old title and the new title are each used as a query: 2,246 queries (1,511 test). Any current version is a correct result; any obsolete member of the lineage is stale.

Change-aware questions. Title queries are artificial, so we also generate natural questions whose answer *changed* between versions. For 300 sampled links, an LLM reads both abstracts and writes a question, the old and new answers, and verbatim evidence from each. Automatic checks reject evidence that is not verbatim, questions that name an RFC or give away the answer, and answers argued from silence. 69 questions pass (23.0%); in 224 cases the model found no substantive change and declined (Table 6 in the appendix).

The benchmark is public on Hugging Face (§9).

3 Q1: does retrieval surface the obsolete version?

final

Setup. Three retrievers: BM25 [7], dense embeddings (`bge-small-en-v1.5` [8]), and their reciprocal rank fusion [9] ("hybrid"). Top 10. With oracle lineage we also test two fixes: `filter` drops obsolete documents; `promote` replaces each obsolete hit with its current version, at the same rank.

Results. Every retriever often ranks the obsolete version first (Fig. 2a): on new-title queries, 42% of the time with BM25, 46% with dense retrieval and 53% with the hybrid; on change-aware questions, 65% [54%,

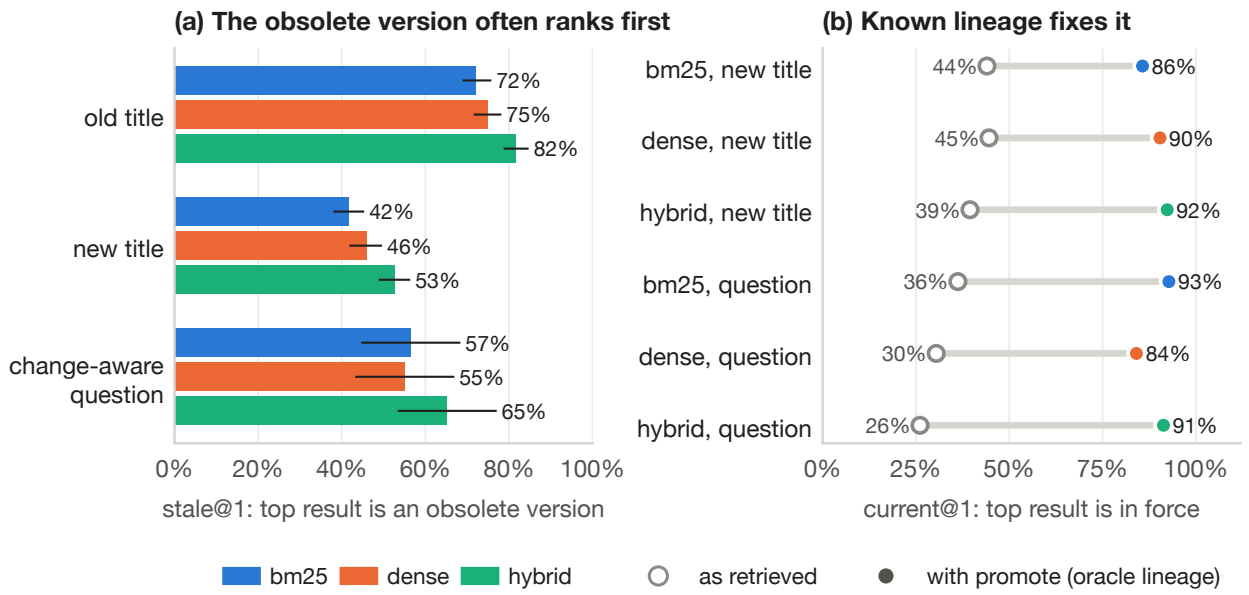


Figure 2. **(a)** Share of queries whose top result is obsolete, by query type and retriever, with 95% intervals. **(b)** Share whose top result is current, as retrieved (hollow) and after `promote` with oracle lineage (filled). Test split; questions use all splits (n=69). Full numbers: Tables 1–2.

77%] with the hybrid. The best topical matcher is the most stale, because the old document *is* a good match for the topic. With oracle lineage, `promote` raises the hybrid's current@1 from 39% to 92% on new titles (Fig. 2b).

> **Takeaway.** Staleness is a property of the collection, not of a weak retriever. Better retrieval makes it worse. Knowing the lineage removes it almost completely.

4 Q2: does it reach the answer?

final · human check of the judge pending

Setup. For each question, the hybrid retriever's results go to an answering model (`claude-sonnet-5-5`, top 5) under three policies, all using oracle lineage except the first: `naive` (context as retrieved); `latest_only` (current versions only); `show_conflict` (`promote` order, obsolete documents kept and marked as replaced). A judge model (`claude-haiku-5-5`) compares each answer with the reference old and new answers. A *stale* outcome gives the old answer without flagging it as outdated.

In the standard context each passage shows its year and RFC number, which a strong model can use to pick the newer one by itself. The *blind* variant hides years and numbers, as in collections without reliable metadata.

Results. With metadata visible, `naive` gives a stale answer 8.7% of the time; the model mostly resolves versions itself. Blind, this rises to 21.7% [13.0%, 31.9%] (Fig. 3). `show_conflict` keeps it at 0.0% and gives the most current answers (88.4% blind), while telling the reader what changed.

Judge reliability. A second model labelled 100 answers without seeing the judge's labels: Cohen's κ = 0.85 [11] (91 of 100 identical). This is agreement between two models; a human check is pending (κ = ??, n = ??).

> **Takeaway.** Where metadata is missing, about one answer in five is stale. Showing both versions, with the old one marked, fixes it, but it needs the lineage.

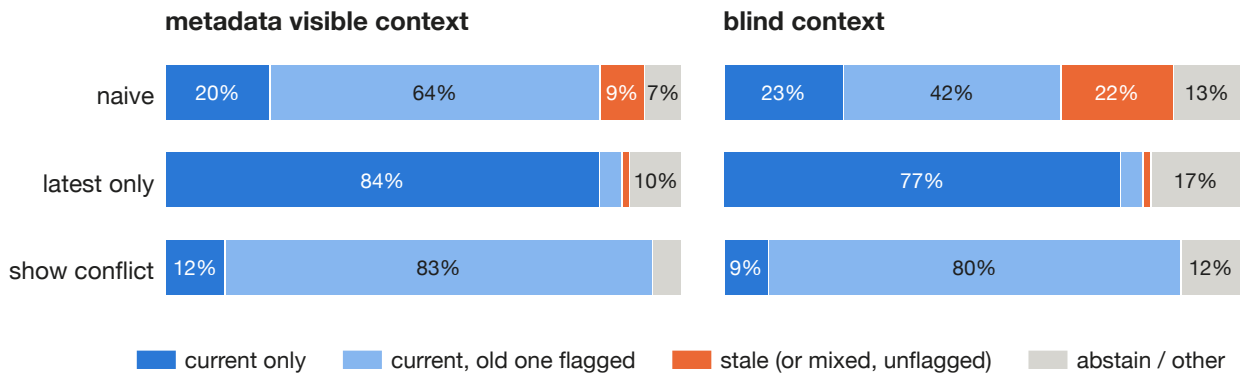


Figure 3. What the answers say, per policy (n=69 questions each). "Current, old one flagged" gives the new answer and mentions the old one as outdated. Table 3 in the appendix gives intervals.

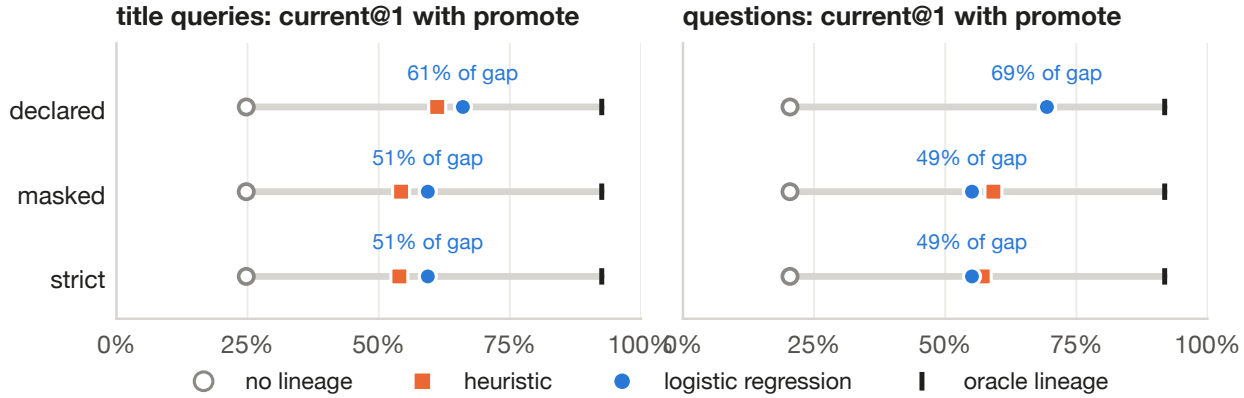


Figure 4. current@1 after `promote` using inferred lineage, between no lineage (hollow) and oracle lineage (bar). Labels: share of the gap closed by the logistic regression. Edge precision, recall and F1 in Table 4.

5 Q3a: inferring lineage cheaply

final · librarian v0

Hiding the answer. Many abstracts say outright "This document obsoletes RFC 2581", and the index's lineage fields are hidden from every method. Three settings: `declared` keeps the text; `masked` replaces every RFC number with a neutral token; `strict` also removes sentences that announce a replacement. `strict` matches collections that never declare it.

Method. Candidate pairs are each document's 10 nearest neighbours by abstract and by title embedding, older first (120,390 pairs, covering 89% of the true links). Each pair gets cheap features (title and abstract similarity, year gap, version cues such as "bis" or "version 2", shared keywords, neighbour rank) and a score from a logistic regression trained on the dev split, with a threshold chosen on dev. Each old document keeps its best replacement above the threshold.

Results. Edge F1 drops from 0.663 (declared) to 0.539 (strict), but downstream the inferred lineage still closes 51% of the gap on title queries (Fig. 4). Its mistakes cluster in a band of high but uncertain scores, mostly pairs with near-identical titles that are companions rather than successors.

> **Takeaway.** Cheap signals recover half of what lineage is worth. The other half sits in ambiguous pairs that need reading, which is the agent's job.

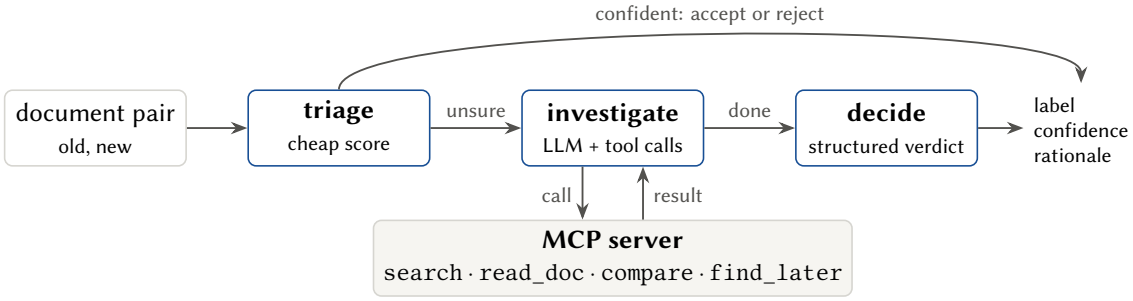


Figure 5. The librarian agent. Only pairs in the cheap scorer's uncertain band reach the language model, so the agent's cost scales with ambiguity, not corpus size.

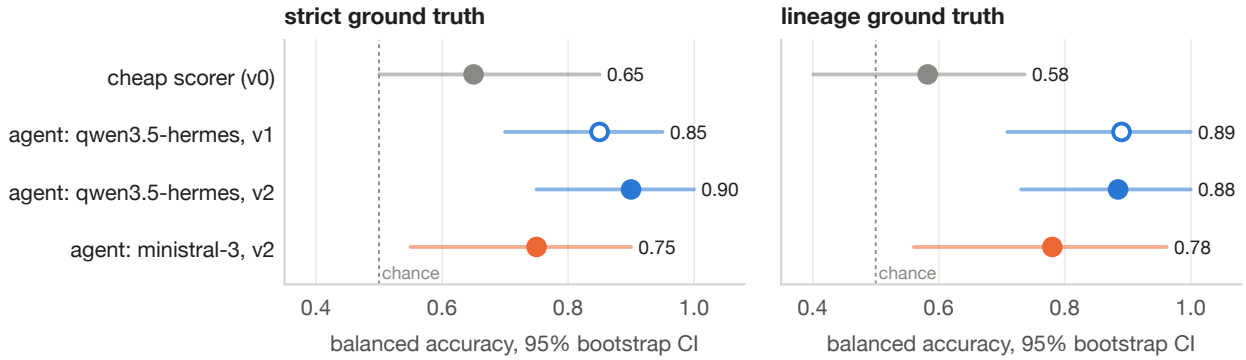


Figure 6. Balanced accuracy on the same 20 band pairs, with 95% stratified bootstrap intervals. Hollow: prompt v1; filled: prompt v2. Blue: Qwen 3.5 (9.7B); orange: Ministral 3 (8B); grey: the cheap scorer.

6 Q3b: a librarian agent for the uncertain cases

preliminary · 20 pairs per run

Design. The agent decides whether a newer document replaces an older one (Fig. 5). It is a LangGraph state machine: *triage* settles pairs the cheap scorer is sure about; *investigate* lets the model call tools for up to 8 steps, ReAct-style [12]; *decide* returns a structured verdict (*supersedes*, *amends*, *conflicts*, *unrelated*) with a confidence and a one-line rationale. The tools come from an MCP server, the same one used in deployment, and never reveal lineage: `search`, `read_doc`, `compare` and `find_later` (documents on the same topic published later: the candidate replacements). Models run locally through Ollama at temperature 0.

Evaluation. We take test pairs whose cheap-scorer probability falls in the uncertain band [0.8, 0.98), `strict` setting, and draw a seeded sample with as many true replacements as non-replacements. The agent and the cheap scorer (with its own threshold, 0.94) are scored *on the same pairs*. Two ground truths: *strict* (the index records the link directly) and *lineage* (a later version through a chain also counts: if A→B→C, then "C replaces A" is right for a reader). Because the sample is balanced and the band is not (only 0.14 of band pairs are true links), the precision expected in use is estimated from the hit rates (Table 5).

Results so far. The cheap scorer reaches 0.65 [0.50, 0.85] on these pairs (Fig. 6). The Qwen agent with prompt v1 reached 0.85, but it never searched: it read both documents, compared them, and accepted look-alike pairs (true-negative rate 0.70). Prompt v2 adds `find_later`, a fixed procedure (compare, then look for later versions) and a confidence rubric; Qwen then checks for later versions on every pair and reaches 0.90 [0.75, 1.00], with true-negative rate 0.90. Ministral 3 with the same prompt reaches 0.75: it called `compare` once per pair and never `find_later`, so it decided from titles and dates.

> **Takeaway.** A small local agent adds value where cheap signals are ambiguous, provided the model actually follows the investigation. With 20 pairs the intervals overlap: this is a direction, not yet a finding. Runs on ≥100 pairs are next.

7 Related work

[TODO: expand; verify every reference before submission.] Time-sensitive QA studies questions whose answer changes over time [2, 3, 4]; we study the document side: which version of a document is in force. Knowledge-conflict work asks how models resolve contradictions between memory and context or between passages [5, 6]; here the conflict has a ground truth, the lineage. LLM judges [10] need validation, which we report as agreement statistics.

8 Limitations and next steps

- One domain (RFCs), with short abstracts. Aviation advisory circulars are next.
- 69 change-aware questions; in some, the reference "new" version has itself been replaced since (flagged in the dataset).
- The judge is checked against a second model, not yet against people.
- Agent results use 20 pairs per run; how much of the remaining gap the agent closes downstream is not yet measured.
- Deployment (MCP and A2A interfaces, a container on Cloud Run) is in progress and will be reported with latency and cost.

9 Reproducibility and release

Code, results and this paper: `https://github.com/CarlHatoum/supersede`. `make paper` regenerates every number, table and figure from `results/`. LLM calls for questions, answers and judging are cached, so `make reproduce` rebuilds them offline; agent decisions are logged per pair with tool traces. The benchmark is at `https://huggingface.co/datasets/hatoum/supersede-rfc`. RFC abstracts ship verbatim, as the IETF Trust Legal Provisions allow unmodified, attributed excerpts but not modified text; the `masked` and `strict` views are derived locally by a script shipped with the data.

A Full tables

retriever	query	policy	stale@1↓	current@1↑	current@10↑	MRR↑
bm25	new title	none	0.417	0.440	0.931	0.628
		filter	0.000	0.750	0.931	0.821
		promote	0.000	0.857	0.983	0.905
	old title	none	0.722	0.152	0.790	0.394
		filter	0.000	0.568	0.790	0.651
		promote	0.000	0.874	0.986	0.916
dense	new title	none	0.459	0.446	0.928	0.643
		filter	0.000	0.803	0.928	0.855
		promote	0.000	0.904	0.992	0.938
	old title	none	0.749	0.170	0.793	0.415
		filter	0.000	0.620	0.793	0.686
		promote	0.000	0.916	0.996	0.947
hybrid	new title	none	0.528	0.395	0.937	0.621
		filter	0.000	0.809	0.937	0.861
		promote	0.000	0.923	0.994	0.950
	old title	none	0.816	0.111	0.810	0.389
		filter	0.000	0.615	0.810	0.689
		promote	0.000	0.927	0.996	0.953

Table 1. Retrieval on test-split title queries (n=727 new-title, 784 old-title). `filter` and `promote` use oracle lineage.

retriever	policy	stale@1↓ [95% CI]	current@1↑
bm25	none	0.565 [0.449, 0.681]	0.362 [0.261, 0.478]
	filter	0.000 [0.000, 0.000]	0.768 [0.667, 0.870]
	promote	0.000 [0.000, 0.000]	0.928 [0.870, 0.986]
dense	none	0.551 [0.435, 0.667]	0.304 [0.203, 0.420]
	filter	0.000 [0.000, 0.000]	0.681 [0.565, 0.783]
	promote	0.000 [0.000, 0.000]	0.841 [0.754, 0.913]
hybrid	none	0.652 [0.536, 0.768]	0.261 [0.159, 0.362]
	filter	0.000 [0.000, 0.000]	0.783 [0.681, 0.870]
	promote	0.000 [0.000, 0.000]	0.913 [0.841, 0.971]

Table 2. Retrieval on change-aware questions (n=69, all splits).

context	policy	stale answer↓ [95% CI]	current↑	conflict flagged	abstain
standard	naive	8.7% [2.9%, 15.9%]	84.1%	63.8%	5.8%
	latest_only	1.4% [0.0%, 4.3%]	88.4%	4.3%	7.2%
	show_conflict	0.0% [0.0%, 0.0%]	94.2%	82.6%	4.3%
blind	naive	21.7% [13.0%, 31.9%]	65.2%	42.0%	13.0%
	latest_only	1.4% [0.0%, 4.3%]	81.2%	4.3%	14.5%
	show_conflict	0.0% [0.0%, 0.0%]	88.4%	79.7%	10.1%

Table 3. Answer outcomes (n=69), hybrid retrieval, top-5 context. "Current" includes answers that flag the old version.

setting	method	edge P	edge R	edge F1	title queries: current@1 (gap)	questions: current@1 (gap)
declared	no lineage	–	–	–	0.248 (0%)	0.204 (0%)
	heuristic	0.325	0.541	0.406	0.612 (54%)	0.694 (69%)
	logreg	0.775	0.578	0.663	0.660 (61%)	0.694 (69%)
	oracle	–	–	–	0.925 (100%)	0.918 (100%)
masked	no lineage	–	–	–	0.248 (0%)	0.204 (0%)
	heuristic	0.288	0.449	0.351	0.543 (44%)	0.592 (54%)
	logreg	0.593	0.481	0.531	0.594 (51%)	0.551 (49%)
	oracle	–	–	–	0.925 (100%)	0.918 (100%)
strict	no lineage	–	–	–	0.248 (0%)	0.204 (0%)
	heuristic	0.286	0.444	0.348	0.539 (43%)	0.571 (51%)
	logreg	0.596	0.492	0.539	0.594 (51%)	0.551 (49%)
	oracle	–	–	–	0.925 (100%)	0.918 (100%)

Table 4. Lineage inference: edge scores on test pairs, and downstream current@1 with `promote` (share of gap closed in brackets).

truth	method	prompt	TPR	TNR	bal. acc. [95% CI]	est. prec.
strict	cheap scorer (v0)	–	0.40	0.90	0.65 [0.50, 0.85]	0.40
	agent: `ministral-3`	v2	0.90	0.60	0.75 [0.55, 0.90]	0.27
	agent: `qwen3.5-hermes`	v1	1.00	0.70	0.85 [0.70, 0.95]	0.35
	agent: `qwen3.5-hermes`	v2	0.90	0.90	0.90 [0.75, 1.00]	0.60
lineage	cheap scorer (v0)	–	0.31	0.86	0.58 [0.40, 0.74]	0.41
	agent: `ministral-3`	v2	0.85	0.71	0.78 [0.56, 0.96]	0.49
	agent: `qwen3.5-hermes`	v1	0.92	0.86	0.89 [0.71, 1.00]	0.68
	agent: `qwen3.5-hermes`	v2	0.77	1.00	0.88 [0.73, 1.00]	1.00

Table 5. Agent and cheap scorer on the same balanced band sample. Estimated precision uses the band's real share of true links (0.14 strict, 0.25 lineage).

outcome	pairs
`skipped_by_model`	224
`accepted`	69
`evidence_current_not_verbatim`	2
`question_leaks_reference`	2
`answer_from_silence`	1
`evidence_stale_not_verbatim`	1
`question_reveals_answer`	1

Table 6. Outcome of question generation per sampled link.

B Agent runs and traces

model	prompt	setting	n	failed	steps	s/pair	labels
qwen3.5-hermes	agent-v1	strict	20	0	3.0	122	amends 2, supersedes 13, unrelated 5
qwen3.5-hermes	agent-v2	strict	20	0	3.0	118	amends 2, supersedes 10, unrelated 8
ministral-3	agent-v2	strict	20	0	2.0	69	amends 2, supersedes 13, unrelated 5

qwen3.5-hermes, agent-v1

- **correct, supersedes**: RFC332 -> RFC342 (gold supersedes; verdict supersedes, confidence 0.95).
  → read_doc({"doc_id": "RFC332"})
  → read_doc({"doc_id": "RFC342"})
  → compare({"doc_a": "RFC332", "doc_b": "RFC342"})
  Both documents share the exact same title ('Network Host Status') and year (1972). In technical standards collections, when two documents with identical titles exist but are distinguished as 'older' and 'newer' (even if the publication year is the same or very close), the newer identifier typically represents a revision or replacement of the older one. The comparison tool confirmed they cover the same topic. Since RFC342 is designated as the 'NEWER' document in the pair, it supersedes RFC332.
- **correct, not**: RFC4145 -> RFC4572 (gold not; verdict unrelated, confidence 0.95).
  → read_doc({"doc_id": "RFC4145"})
  → read_doc({"doc_id": "RFC4572"})
  → compare({"doc_a": "RFC4145", "doc_b": "RFC4572"})
  RFC4145 (2005) defines media transport over plain TCP, while RFC4572 (2006) defines media transport over TLS. Although both use SDP and share similar attribute names ('setup', 'connection'), they address different underlying protocols (TCP vs. TLS). RFC4572 does not replace or supersede RFC4145; instead, they are complementary standards for different security requirements. Therefore, they are unrelated in the sense that neither supersedes the other.
- **error**: RFC266 -> RFC370 (gold not; verdict supersedes, confidence 0.95).
  → read_doc({"doc_id": "RFC266"})
  → read_doc({"doc_id": "RFC370"})
  → compare({"doc_a": "RFC266", "doc_b": "RFC370"})
  The documents share identical titles ('Network host status' vs 'Network Host Status') and cover the same topic. The newer document (RFC370, 1972) is a direct update to the older one (RFC266, 1971). In technical standards, when a new version with the same title replaces an older one, it supersedes the previous version.

qwen3.5-hermes, agent-v2

- **correct, supersedes**: RFC3016 -> RFC6416 (gold supersedes; verdict supersedes, confidence 0.95).
  → compare({"doc_a": "RFC3016", "doc_b": "RFC6416"})
  → find_later({"doc_id": "RFC3016", "k": 5})
  RFC6416 explicitly identifies itself as a 'necessary revision' of RFC3016 in its abstract. It states that it updates and completes the specification while discussing backward compatibility with the older document. The scope (RTP Payload Format for MPEG-4 Audio/Visual Streams) is identical, and the newer document corrects misalignments found in the older one, which is the definition of superseding in this context.
- **correct, not**: RFC4145 -> RFC4572 (gold not; verdict unrelated, confidence 0.95).
  → compare({"doc_a": "RFC4145", "doc_b": "RFC4572"})
  → find_later({"doc_id": "RFC4145", "k": 5})

RFC4145 defines media transport over plain TCP, while RFC4572 defines media transport over TLS (Transport Layer Security). Although both are SDP extensions for connection-oriented transport, they address different underlying protocols with different security properties. One does not replace the other; instead, they serve as companion specifications allowing implementers to choose between unencrypted and encrypted transport mechanisms based on their needs. The fact that RFC8122 (a later update to the TLS specification) exists further confirms that RFC4572 was an early version of the TLS SDP spec, not a successor to the plain TCP spec.

- **error**: `RFC332 -> RFC342` (gold supersedes; verdict `unrelated`, confidence 0.95).
  → `compare({"doc_a": "RFC332", "doc_b": "RFC342"})`
  → `find_later({"doc_id": "RFC332", "k": 5})`
  Both documents share the exact same title ('Network Host Status') and were published in the same year (1972). The comparison tool indicates no abstract overlap (both lack abstracts) but 100% title word overlap. The 'find_later' query for RFC332 returned other RFCs from 1972, none of which were RFC342, suggesting they are not in a direct versioning chain where one replaces the other. Documents with identical titles from the same era often represent companion documents (e.g., different profiles or specific aspects) rather than a successor relationship. Since neither clearly restates and revises the scope of the other to act as a replacement, and they appear to be parallel works on the same topic, they are classified as unrelated in terms of supersession.

`ministral-3`, agent-v2

- **correct, supersedes**: `RFC3016 -> RFC6416` (gold supersedes; verdict `supersedes`, confidence 0.95).
  → `compare({"doc_a": "RFC3016", "doc_b": "RFC6416"})`
  RFC6416 explicitly states it is a 'necessary revision' of RFC3016, summarizes changes, and addresses backward compatibility issues. The scope remains identical (RTP payload formats for MPEG-4 Audio/Visual streams), and the newer document restates/revises the older one's content while clarifying limitations and correcting misalignments. This is a direct replacement, not merely an extension or companion document.

- **correct, not**: `RFC4145 -> RFC4572` (gold not; verdict `amends`, confidence 0.9).
  → `compare({"doc_a": "RFC4145", "doc_b": "RFC4572"})`
  RFC4572 explicitly states it 'extends and updates' a prior reference (likely RFC4145) by adding TLS security to TCP-based media transport in SDP. The newer document introduces a new protocol identifier ('TCP/TLS') and a fingerprint attribute, but it does not restate or revise the entire scope of RFC4145. Instead, it adds a new mechanism (TLS) to the same transport framework. The older document remains valid for plain TCP transport, and the newer one complements it rather than replacing it entirely. This is a clear case of amendment, not supersession.

- **error**: `RFC332 -> RFC342` (gold supersedes; verdict `unrelated`, confidence 0.0).
  → `compare({"doc_a": "RFC332", "doc_b": "RFC342"})`
  RFC332 and RFC342 share the same title and year (1972), meaning neither is newer than the other. Since supersession requires a later document to replace an earlier one, they cannot be in a superseding relationship. The lack of a chronological gap or explicit revision history makes this a definitive conclusion.

C Provenance

section	source (under `results/` unless stated)
Retrieval staleness (title queries)	`rfc_staleness_test.json`
Retrieval staleness (change-aware questions)	`rfc_questions_all.json`
Question generation	`data/rfc/questions_manifest.json`
Answer staleness (standard)	`rfc_answer_staleness_hybrid.json`
Answer staleness (blind)	`rfc_answer_staleness_hybrid_blind.json`
Judge agreement (model rater)	`rfc_answer_staleness_hybrid_labels_claude-opus-5-5.csv`
Lineage inference (librarian v0)	`rfc_lineage_inference.json`
Agent: qwen3.5-hermes, agent-v1	`agent/rfc_agent_strict_ollama-edtorre-qwen3-5-hermes_agent-v1.json`
Agent: qwen3.5-hermes, agent-v2	`agent/rfc_agent_strict_ollama-edtorre-qwen3-5-hermes_agent-v2.json`
Agent: ministral-3, agent-v2	`agent/rfc_agent_strict_ollama-ministral-3-8b_agent-v2.json`

References

[1] P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 2020.

[2] W. Chen, X. Wang, W. Y. Wang. A dataset for answering time-sensitive questions. *NeurIPS Datasets and Benchmarks*, 2021.

[3] J. Kasai et al. RealTime QA: What's the answer right now? *NeurIPS Datasets and Benchmarks*, 2023.

[4] B. Dhingra et al. Time-aware language models as temporal knowledge bases. *TACL*, 2022.

[5] S. Longpre et al. Entity-based knowledge conflicts in question answering. *EMNLP*, 2021.

[6] J. Xie et al. Adaptive chameleon or stubborn sloth: Revealing the behavior of large language models in knowledge conflicts. *ICLR*, 2024.

[7] S. Robertson, H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in IR*, 2009.

[8] S. Xiao et al. C-Pack: Packaged resources to advance general Chinese embedding. arXiv:2309.07597, 2023.

[9] G. V. Cormack, C. L. A. Clarke, S. Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. *SIGIR*, 2009.

[10] L. Zheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets and Benchmarks*, 2023.

[11] J. Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 1960.

[12] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.